

DEPARTAMENTO:	Ciencia de la computación	CARRERA:	ITIN		
ASIGNATURA:	Fundamentos Web	NIVEL:	4	FECHA:	19/7/2026
DOCENTE:	GEOVANNY JOSÉ BRITO CASANOVA	PRÁCTICA N°:	2	CALIFICACIÓN:	19

Manipulación de elementos web

Elkin Alejandro Vera Gorozabel

RESUMEN

En este laboratorio se desarrolló un sistema de gestión de estudiantes mediante JavaScript y la manipulación del DOM. La información de cada registro se almacenó en un arreglo global, el cual sirve como fuente principal para actualizar la tabla de forma dinámica. Además, se incorporó la visualización del JSON individual y general utilizando un modal Bootstrap junto con JSON.stringify().

El sistema permite editar registros cargando nuevamente los datos en el formulario y diferenciando entre el modo de registro y el modo de edición. También se integró la eliminación de estudiantes con una confirmación mediante SweetAlert2, garantizando mayor seguridad y UX. Se añadió un buscador capaz de localizar estudiantes por nombre, carrera o semestre recorriendo el arreglo y comparando los datos con includes(). Acompañado de un filtro por carrera que actualiza la tabla según la opción seleccionada, manteniendo siempre sincronizada la información mostrada con los datos almacenados en memoria

Palabras Claves: DOM, SweetAlert2, CRUD Java Script

1. REPOSITORIO

<https://github.com/ElkinVera17/registroEstudiantesJS.git>

2. INDICE DE CONTENIDO

1. Contenido

1.	REPOSITORIO	1
2.	INDICE DE CONTENIDO	1
3.	INDICE DE FIGURA	2
4.	INDICE DE CODIGO	2
5.	INTRODUCCIÓN:	3
6.	OBJETIVO(S):	3
7.	MARCO TEÓRICO:	4
8.	DESCRIPCIÓN DEL PROCEDIMIENTO:	6
9.	ANÁLISIS DE RESULTADOS:	13

10.	GRÁFICOS O FOTOGRAFÍAS:	14
11.	DISCUSIÓN:	18
12.	CONCLUSIONES:	18
13.	BIBLIOGRAFÍA:	19
14.	Criterio personal	19

3. INDICE DE FIGURA

Figura 1	Objeto cursos	6
Figura 2	Nueva columna acciones	7
Figura 3	Botones	8
Figura 4	SweetAlert eliminar?	11
Figura 5	SweetAlert2 Registrado	14
Figura 6	Estudiantes registrados	14
Figura 7	JSON individual	15
Figura 8	JSON cursos	15
Figura 9	Editar	16
Figura 10	Sweetalert2 editar	16
Figura 11	Tabla editada	16
Figura 12	sweetalert2 eliminar?	16
Figura 13	Eliminado de tabla	17
Figura 14	Actualiza contador	17
Figura 15	sweetalert reinicio	17
Figura 16	Limpieza total	18

4. INDICE DE CODIGO

Código 1	getElementById	4
Código 2	innerHTML	4
Código 3	createElement	5
Código 4	appendChild	5
Código 5	textContent	5
Código 6	className	5
Código 7	classList	5
Código 8	JSON.stringify	6
Código 9	Inicializa Arreglo global	6
Código 10	Asigna valores al arreglo	6
Código 11	Una fila por cada estudiante	6
Código 12	Nueva columna en la tabla	7
Código 13	Creación de botones	7

Código 14Modal de bootstrap	8
Código 15Mostrar JSON en modal.....	8
Código 16 JSON general	9
Código 17 Mensaje con SweetAlert2.....	9
Código 18 Editar Estudiante	9
Código 19Cambiar texto al botón.....	9
Código 20 Modo Edición	10
Código 21 Confirmación de edición.....	10
Código 22 Eliminar estudiante	10
Código 23 Mensaje de confirmación	10
Código 24 Cantidad de estudiantes	11
Código 25 Recargar	11
Código 26 Mostrar alert	12
Código 27 Buscador.....	12
Código 28 Filtrar con select	13

5. INTRODUCCIÓN:

En este laboratorio se desarrollo un sistema de registro de estudiantes utilizando JavaScript y manipulación del DOM, haciendo un CRUD volátil (almacenar, mostrar, editar, eliminar y filtrar) de la información de los estudiantes de forma en una tabla HTML.

Durante su desarrollo se aplicaron nuevos conceptos como el manejo de arreglos de objetos para almacenar los datos, el uso de eventos y funciones para actualizar la interfaz en tiempo real, la implementación de modales de Bootstrap para visualizar información en formato JSON, y la librería SweetAlert2 para mostrar alertas y confirmaciones visuales al usuario. Además se agregó la búsqueda y el filtrado de registros mediante la comparación de datos dentro del arreglo,

6. OBJETIVO(S):

6.1 General:

- Ampliar el sistema de registro de estudiantes desarrollado en clase mediante la manipulación del DOM modales de Bootstrap y la librería SweetAlert2 para crear una pagina web con un CRUD dinámico que garantiza la UX

6.2 Especificos

- Implementar el registro de estudiantes en un arreglo de objetos y reflejar dicha información dinámicamente en la tabla mediante manipulación del DOM.
- Diseñar un modal de Bootstrap que muestre la información de cada estudiante y del arreglo completo en formato JSON.
- Desarrollar las funciones de edición y eliminación de estudiantes, actualizando la tabla, el contador y el arreglo tras cada acción.
- Incorporar un buscador y un filtro por carrera, e integrar la librería SweetAlert2 para mejorar los mensajes de confirmación y notificación del sistema.

7. MARCO TEÓRICO:

7.1 Document Object Model (DOM)

El Document Object Model (DOM) es una interfaz de programación de aplicaciones (API) neutral para documentos HTML y XML que representa la estructura del documento como un árbol jerárquico de nodos de objetos, permitiendo a los lenguajes de programación acceder, estructurar y modificar de manera dinámica el contenido, estilo y estructura de una página web (Mozilla Contributors, 2024)

Para interactuar de forma programática con la interfaz, JavaScript utiliza métodos de acceso específicos como `document.getElementById()`, el cual localiza de forma eficiente un nodo único mediante su identificador en un tiempo de búsqueda constante (Flanagan, 2011). Una vez seleccionado el elemento, es posible modificar su contenido mediante propiedades como `innerHTML`

7.2 Eventos en JavaScript

Un evento es un objeto que representa una señal emitida por el entorno del navegador para notificar al motor de JavaScript que ha sucedido una interacción del usuario, un cambio de estado del sistema o una operación de red asíncrona (Haverbeke, 2018). Al estar basado en un bucle de eventos no bloqueante, la aplicación web espera pasivamente estas señales; para reaccionar ante ellas, se registran funciones controladoras (listeners) sobre nodos específicos del DOM, los cuales encolan y procesan las respuestas de forma ordenada y asíncrona (Mozilla Contributors, 2024).

Durante el desarrollo de interfaces, los eventos como click, submit e input son esenciales para habilitar la interacción entre el usuario y la aplicación. El evento click responde al dispositivo apuntador sobre un elemento de acción (Mozilla Contributors, 2024)

7.3 `getElementById`

Que se ejecuta es lee formulario y se in ta a leer el valor del campo `<input id=fechas y a es enla página:`

Código 1 `getElementById`

```
1. document.getElementById("output").innerHTML = La fecha seleccionada es:  
2. `${document.getElementById('fecha').value}`;
```

Para leer el dato contenido en el formulario, accedemos a su valor tras haberlo identificado con `getElementById` (Alessandra Salvaggio, 2019).

Es decir, gracias a esta función obtenemos todo un elemento mediante su id desde el html a nuestro javaScrib para ser manipulada

7.4 `innerHTML`

El DOM HTML permite que JavaScript cambie tanto el texto como el contenido de los elementos HTML. La forma más sencilla de modificar el contenido es usando la propiedad: `innerHTML`

Código 2 `innerHTML`

```
1. document.getElementById(id).innerHTML = new HTML
```

La propiedad puede usarse para obtener o cambiar cualquier elemento HTML, incluyendo y `.innerHTML<html><body>`

métodos

7.5 createElement

El método createElement() expuesto por la interfaz Document se encarga de instanciar un nuevo elemento HTML que posee el nombre local de la etiqueta especificada por parámetro (Mozilla Developer Network, 2024)

Código 3 createElement

```
1. const nuevoDiv = document.createElement("div");
```

7.6 appendChild

Añade un objeto nodo determinado al final de la lista de elementos hijos que dependen directamente de un nodo padre especificado (Mozilla Developer Network, 2024)

Código 4 appendChild

```
1. const parrafoHijo = document.createElement("p");  
2. document.body.appendChild(parrafoHijo);
```

7.7 textContent

La propiedad textContent perteneciente a la interfaz Node representa o altera el contenido textual de un nodo y de todos sus descendientes jerárquicos (Mozilla Developer Network, 2024)

Código 5 textContent

```
1. const contenedor = document.createElement("section");  
2. contenedor.textContent = "Este es un bloque de texto que no interpretará etiquetas  
<strong>HTML</strong>.";
```

7.8 className

La propiedad className de la interfaz Element se encarga de leer y definir directamente el valor asignado al atributo global de clases (class) del elemento en cuestión (Mozilla Developer Network, 2024)

Código 6 className

```
1. const cajaDeAlerta = document.createElement("div");  
2. cajaDeAlerta.className = "alerta alerta-error visible";
```

7.9 classList

La propiedad de solo lectura classList de la interfaz Element expone una colección activa de tipo DOMTokenList que representa de forma desglosada y estructurada cada una de las clases que posee el elemento en su atributo de clase (Mozilla Developer Network, 2024)

Código 7 classList

```
1. const elementoMenu = document.createElement("nav");  
2. elementoMenu.classList.add("menu-navegacion", "sombreado");  
3. elementoMenu.classList.toggle("colapsado");
```

7.10 JSON.stringify

El método estático `JSON.stringify()` convierte un valor, variable u objeto de JavaScript en una cadena de caracteres estructurada que cumple de manera rigurosa con la sintaxis de la Notación de Objetos de JavaScript (JSON) (Mozilla Developer Network, 2024).

Código 8 `JSON.stringify`

```
1. const datosUsuario = { id: 101, nombres: "Carlos", activo: true, sesion: undefined };
2. const cadenaDeSalida = JSON.stringify(datosUsuario);
```

8. DESCRIPCIÓN DEL PROCEDIMIENTO:

7.11 Crear un arreglo global para almacenar los estudiantes registrados. Este arreglo será la fuente principal de datos del sistema y permitirá conservar temporalmente la información ingresada desde el formulario.

Código 9 Inicializa Arreglo global

```
1. let cursos = [];
```

Descripción: Se creo una variable global de tipo arreglo para almacenar todos los estudiantes

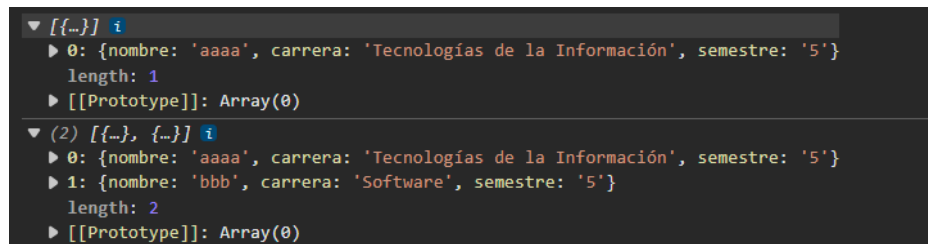
7.12 Guardar cada estudiante agregado dentro del arreglo. Cuando el usuario complete el formulario y presione el botón de registro, el estudiante debe almacenarse como un objeto con nombre, carrera y semestre.

Código 10 Asigna valores al arreglo

```
1. cursos.push(estudiante);
```

Descripción: Se almacena cada estudiante en cursos mediante la función push

Figura 1 Objeto cursos



Descripción: Mediante `console.log()` comprobar el contenido de la variable global

7.13 Actualizar la lógica de la tabla para que los datos registrados se muestren desde el arreglo. La tabla debe reflejar visualmente los estudiantes almacenados.

Código 11 Una fila por cada estudiante

```
1. function agregarFilaTabla() {
2.   const cuerpoTabla = document.getElementById("cuerpoTabla");
3.   cuerpoTabla.innerHTML = "";
4.
5.   cursos.forEach((estudiante, indice) => {
6.     const fila = document.createElement("tr");
7.
8.     const columnaNumero = document.createElement("td");
9.     columnaNumero.textContent = indice + 1;
10.  }
```

```

11.     const columnaNombre = document.createElement("td");
12.     columnaNombre.textContent = estudiante.nombre;
13.
14.     const columnaCarrera = document.createElement("td");
15.     columnaCarrera.textContent = estudiante.carrera;
16.
17.     const columnaSemestre = document.createElement("td");
18.     columnaSemestre.textContent = estudiante.semestre;
19.
20.     fila.appendChild(columnaNumero);
21.     fila.appendChild(columnaNombre);
22.     fila.appendChild(columnaCarrera);
23.     fila.appendChild(columnaSemestre);
24.
25.     cuerpoTabla.appendChild(fila);
26.   });
27. }

```

Descripción: El forEach y con la función lambda dibuja la tabla de acuerdo al arreglo curso primero limpia la tabla para por cada estudiante ingresado en cursos se cree una fila

7.14 Agregar una columna de acciones en la tabla. Esta columna permitirá colocar botones para consultar, editar y eliminar cada registro.

Código 12 Nueva columna en la tabla

```
1. const columnaAcciones = document.createElement("td");
```

Descripción: Mediante document.createElement editamos el DOM para crear un elemento de tipo td o columna acciones

Figura 2 Nueva columna acciones

#	Nombre	Carrera	Semestre	Acciones
---	--------	---------	----------	----------

Descripción: Se visualiza la nueva columna de acciones agregada a cada fila de la tabla de estudiantes.

7.15 Crear un botón para ver el JSON individual de cada estudiante. Al presionarlo, se debe mostrar en un modal la información del estudiante seleccionado.

Código 13 Creación de botones

```

.     const ver = document.createElement("button");
3.     ver.textContent = "ver";
4.     const editar = document.createElement("button");
5.     editar.textContent = "editar";
6.     const eliminar = document.createElement("button");
7.     eliminar.textContent = "eliminar";
8.
9.     ver.className = "btn btn-info btn-sm";
10.    editar.className = "btn btn-warning btn-sm mx-1";
11.    eliminar.className = "btn btn-danger btn-sm";

```

```

1. columnaAcciones.appendChild(ver);
2. columnaAcciones.appendChild(editar);
3. columnaAcciones.appendChild(eliminar);
4. fila.appendChild(columnaAcciones);

```

Descripción: Se crean los botones ver, editar y eliminar, se les asignan las clases de Bootstrap y se agregan a la columna de acciones de la fila.

Figura 3 Botones

#	Nombre	Carrera	Semestre	Acciones
1	aa	Tecnologías de la Información	4	ver editar eliminar

Descripción: Se muestran los botones ver, editar y eliminar ya integrados en la tabla de estudiantes.

7.16 Crear un modal Bootstrap para visualizar información en formato JSON. El modal debe abrirse desde JavaScript y mostrar los datos de forma ordenada.

Código 14 Modal de bootstrap

```

1. <!--modal bootstrap -->
2. <div class="modal fade" id="modalJSON" tabindex="-1">
3.   <div class="modal-dialog">
4.     <div class="modal-content">
5.
6.       <div class="modal-header">
7.         <h5 class="modal-title">Información del estudiante</h5>
8.         <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
9.       </div>
10.
11.       <div class="modal-body">
12.         <pre id="contenidoJSON"></pre>
13.       </div>
14.
15.       <div class="modal-footer">
16.         <button class="btn btn-secondary" data-bs-dismiss="modal">
17.           Cerrar
18.         </button>
19.       </div>
20.
21.     </div>
22.   </div>
23. </div>
  
```

Descripción: Se define la estructura del modal de Bootstrap donde se mostrará la información del estudiante en formato JSON.

7.17 Crear una función para abrir el modal y cargar la información correspondiente. Esta función debe recibir un estudiante o el arreglo completo y convertirlo a texto JSON mediante JSON.stringify.

Código 15 Mostrar JSON en modal

```

1. function mostrarJSON(estudiante) {
2.   const textoJSON = JSON.stringify(estudiante, null, 4);
3.   document.getElementById("contenidoJSON").textContent = textoJSON;
4.   const modal = new bootstrap.Modal(document.getElementById("modalJSON"));
5.   modal.show();
6. }
  
```

Descripción: La función convierte el estudiante recibido a texto JSON con JSON.stringify y lo despliega dentro del modal.

7.18 Agregar un botón para ver el JSON general. Este botón debe mostrar en el modal todos los estudiantes registrados en el arreglo.

Código 16 JSON general

```
1. btnJSONGeneral.addEventListener("click", mostrarJSONGeneral);
2.
   const textoJSON = JSON.stringify(cursos, null, 4);
4. document.getElementById("contenidoJSON").textContent = textoJSON;
5. const modal = new bootstrap.Modal(document.getElementById("modalJSON"));
6. modal.show();
```

Descripción: El evento del botón de JSON general convierte todo el arreglo cursos a JSON y lo muestra en el modal.

7.19 Validar que el JSON general muestre un mensaje adecuado cuando todavía no existan estudiantes registrados.

7.20

Código 17 Mensaje con SweetAlert2

```
1. if(cursos.length === 0) {
2.     Swal.fire({
3.         icon: "info",
4.         title: "Sin registros",
5.         text: "No existen estudiantes registrados."
6.     });
7. }else
```

Descripción: Se valida con SweetAlert2 que exista al menos un estudiante antes de generar el JSON general.

7.21 Agregar la opción de editar estudiante. Al presionar editar, los datos del estudiante deben cargarse nuevamente en el formulario. E implementar un modo de edición. El sistema debe diferenciar si el usuario está agregando un estudiante nuevo o modificando uno existente.

Código 18 Editar Estudiante

```
1. function editarEstudiante(indice) {
2.     const estudiante = cursos[indice];
3.     document.getElementById("nombre").value = estudiante.nombre;
4.     document.getElementById("carrera").value = estudiante.carrera;
5.     document.getElementById("semestre").value = estudiante.semestre;
6.     modoEdicion = true;
7.     indiceEditar = indice;
8. }
```

Descripción: La función carga en el formulario los datos del estudiante seleccionado y activa el modo de edición guardando su índice.

7.22 Cambiar el texto del botón principal durante la edición. Cuando el usuario esté editando, el botón debe indicar que guardará cambios; al finalizar, debe regresar a su estado inicial.

Código 19 Cambiar texto al botón

```
1. btnAgregar.textContent = "Guardar cambios";
```

Descripción: Se cambia el texto del botón principal a "Guardar cambios" mientras el sistema se encuentra en modo edición.

7.23 Actualizar la tabla después de editar un estudiante. Los cambios deben reflejarse en la tabla y en el arreglo de estudiantes.

Código 20 Modo Edición

```
1. if (modoEdicion) {
2.     cursos[indiceEditar] = estudiante;
3.     modoEdicion = false;
4.     indiceEditar = -1;
5.     btnAgregar.textContent = "Agregar estudiante";
6.
7. } else {
```

Descripción: Si el modo de edición está activo, se actualiza el estudiante en el arreglo y el botón regresa a su estado inicial.

Código 21 Confirmación de edición

```
1. if (modoEdicion) {
2.     Swal.fire({
3.         title: "¿Guardar cambios?",
4.         text: "Se actualizará la información de este estudiante.",
5.         icon: "warning",
6.         showCancelButton: true,
7.         confirmButtonText: "Guardar",
8.         cancelButtonText: "Cancelar"
9.     }).then((resultado) => {
10.        if (resultado.isConfirmed) {
11.            cursos[indiceEditar] = estudiante;
12.            modoEdicion = false;
13.            indiceEditar = -1;
14.            btnAgregar.textContent = "Agregar estudiante";
15.            mostrarMensaje("Estudiante actualizado correctamente.", "success");
16.        }
17.    });
18. }
```

Descripción: Se agrega una confirmación con SweetAlert2 antes de guardar los cambios de edición del estudiante.

7.24 Agregar la opción de eliminar estudiante. Al eliminar, el registro debe quitarse del arreglo y de la tabla.

Código 22 Eliminar estudiante

```
1. cursos.splice(indice, 1); //elimina(pocision,cantidad)
2. actualizarTotal();
3. mostrarMensaje("Estudiante eliminado correctamente.", "success");
4. agregarFilaTabla();
```

Descripción: El estudiante seleccionado se elimina del arreglo con splice y se actualizan el contador, el mensaje y la tabla.

7.25 Confirmar la eliminación antes de borrar un registro. El estudiante debe aplicar una confirmación visual para evitar eliminaciones accidentales.

Código 23 Mensaje de confirmación

```
1. function eliminarEstudiante (indice){
2.     const estudiante = cursos[indice];
3.     Swal.fire({
4.         title: "¿Eliminar estudiante?",
5.         text: `Se eliminara el estudiante ${estudiante.nombre} de la lista.`,
6.         icon: "warning",
7.         showCancelButton: true,
8.         confirmButtonText: "Eliminar",
9.         cancelButtonText: "Cancelar"
10.    }).then((resultado) => {
```

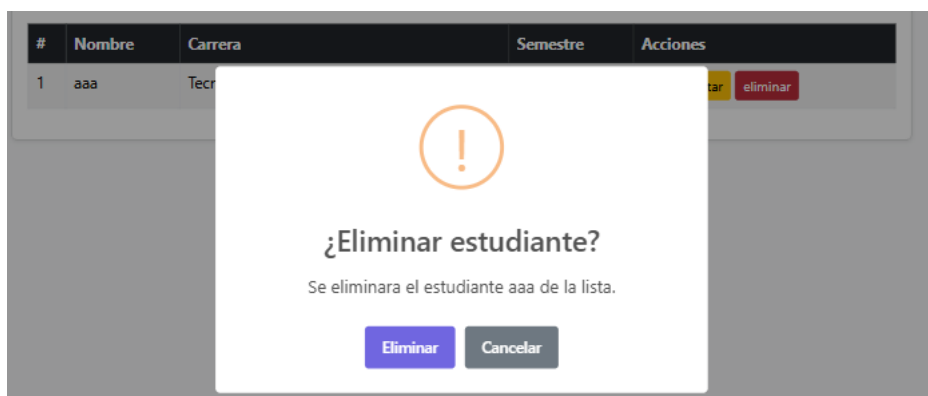
```

11.   if (resultado.isConfirmed) {
12.     cursos.splice(indice, 1);//elimina(pocision,cantidad)
13.     actualizarTotal();
14.     mostrarMensaje("Estudiante eliminado correctamente.", "success");
15.     agregarFilaTabla();
16.   }
17. });
18. }
19.

```

Descripción: Se muestra una confirmación con SweetAlert2 antes de eliminar definitivamente al estudiante seleccionado.

Figura 4 SweetAlert eliminar?



Descripción: Se visualiza la ventana de SweetAlert2 solicitando confirmación para eliminar un estudiante.

7.26 Actualizar el contador de estudiantes. El contador debe mostrar la cantidad real de estudiantes registrados después de agregar, editar, eliminar o limpiar.

Código 24 Cantidad de estudiantes

```

1. function actualizarTotal() {
2.   document.getElementById("totalEstudiantes").textContent = cursos.length;
3. }

```

Descripción: La función actualiza el contador mostrando la cantidad total de estudiantes almacenados en cursos.

7.27 Limpiar tabla, arreglo y formulario. El botón de limpieza debe reiniciar la información visual, el arreglo de estudiantes, el contador y cualquier estado de edición activo.

Código 25 Recargar

```

1. function reinicio(){
2.   swal.fire({
3.     title: "¿Eliminar todo?",
4.     text: "Se perderan todos los datos ingresados.",
5.     icon: "warning",
6.     showCancelButton: true,
7.     confirmButtonText: "Reiniciar",
8.     cancelButtonText: "Cancelar"
9.   }).then((resultado) => {
10.    if (resultado.isConfirmed) {
11.      cursos = [];
12.      actualizarTotal();
13.      limpiarTabla();
14.      limpiarFormulario();
15.    }

```

```
16.   })  
17. }
```

Descripción: Se confirma con SweetAlert2 antes de reiniciar el arreglo, la tabla, el formulario y el contador.

7.28Mostrar mensajes visuales al usuario. El sistema debe informar cuando un estudiante sea agregado, editado, eliminado o cuando exista una validación pendiente.

Código 26 Mostrar alert

```
1. function mostrarMensaje(texto, tipo) {  
2.   const mensaje = document.getElementById("mensaje");  
3.   mensaje.className = `alert alert-${tipo} mt-3`;  
4.   mensaje.textContent = texto;  
5. }
```

Descripción: La función muestra un mensaje de alerta con el texto y el tipo recibidos como parámetros.

7.29Agregar un input de búsqueda y un botón “Buscar”. Al escribir un nombre, carrera o semestre, la tabla debe mostrar solo los estudiantes que coincidan con el texto ingresado. La búsqueda debe hacerse sobre el arreglo de estudiantes registrados y luego volver a cargar la tabla con los resultados encontrados. También se debe agregar un botón “Limpiar búsqueda” para borrar el texto escrito y volver a mostrar todos los estudiantes registrados. El estudiante debe documentar cómo capturó el valor del input, cómo filtró el arreglo y cómo actualizó nuevamente la tabla.

Código 27 Buscador

```
1. function buscarEstudiante() {  
2.   const texto = document.getElementById("txtBuscar").value.trim().toLowerCase();  
3.   let encontrados = [];  
4.  
5.   if (texto === "") {  
6.     Swal.fire({  
7.       icon: "warning",  
8.       title: "Buscador vacio",  
9.       text: "Ingrese un campo para filtrar"  
10.    });  
11.   }else{  
12.  
13.     for (let i = 0; i < cursos.length; i++) {  
14.       let estudiante = cursos[i];  
15.  
16.       if (//truo o falso  
17.         estudiante.nombre.toLowerCase().includes(texto)  
18.         || estudiante.carrera.toLowerCase().includes(texto) ||estudiante.semestre.toString().includes(texto)  
19.       ) {  
20.         encontrados.push(estudiante);  
21.       }  
22.       agregarFilaTabla(encontrados);  
23.     }  
24.     if (encontrados.length === 0) {  
25.       Swal.fire({  
26.         icon: "info",  
27.         title: "Sin resultados",  
28.         text: "No se encontraron estudiantes."  
29.       });  
30.     }  
31.   }
```

Descripción: La función filtra el arreglo cursos según el texto ingresado y vuelve a dibujar la tabla solo con los resultados encontrados.

7.30 Agregar un filtro por carrera mediante un select adicional, donde se carguen las opciones de carrera disponibles y un botón “Filtrar”; al seleccionar una carrera, la tabla debe mostrar solo los estudiantes que pertenezcan a esa carrera, y si se selecciona la opción “Todas”, deben volver a mostrarse todos los estudiantes registrados. La búsqueda debe realizarse sobre el arreglo de estudiantes, no sobre las filas ya dibujadas en la tabla, y el estudiante deberá explicar cómo tomó el valor seleccionado, cómo comparó ese valor con la carrera de cada estudiante y cómo actualizó nuevamente la tabla con los resultados filtrados.

Código 28 Filtrar con select

```

1. function filtrarCarrera(){
2.   const carrera = document.getElementById("selectCarrera").value;
3.   let encontrados = [];
4.
5.   if (carrera === "Todas") {
6.     Swal.fire({
7.       icon: "info",
8.       title: "Sin filtro",
9.       text: "Usted esta filtrado con todas las carreras"
10.    });
11.  }else{
12.
13.    for (let i = 0; i < cursos.length; i++) {
14.      let estudiante = cursos[i];
15.
16.      if (estudiante.carrera === carrera){
17.        encontrados.push(estudiante);
18.      }
19.    }
20.
21.  }
22.  agregarFilaTabla(encontrados);
23. }
  
```

Descripción: La función filtra el arreglo cursos según la carrera seleccionada en el select y actualiza la tabla con los resultados.

9. ANÁLISIS DE RESULTADOS:

El arreglo cursos actuó en todo momento como la única fuente de verdad del sistema: la tabla, el contador, el JSON y los filtros nunca mostraron información propia, sino que siempre la leyeron y reconstruyeron a partir de ese arreglo. Esa separación entre datos y vista es la que evitó inconsistencias durante las pruebas de edición y eliminación.

Funcionalidad probada	Resultado esperado	Resultado obtenido	Observación
Registro de estudiante	Se agrega al arreglo y aparece en la tabla	Correcto	La tabla nunca se editó manualmente, siempre se redibujó desde el arreglo
JSON individual / general	Muestra los datos exactos del arreglo	Correcto	Confirmó que JSON.stringify no altera ni pierde información
Edición	El registro se actualiza sin duplicarse	Correcto	Dependió de guardar bien el índice Editar antes de modificar
Eliminación	El registro desaparece de tabla y arreglo	Correcto	El splice() dejó el arreglo sin "huecos", a diferencia de solo ocultar la fila
Búsqueda por texto	Filtra por coincidencia parcial	Correcto	Tolerante a mayúsculas gracias a toLowerCase()

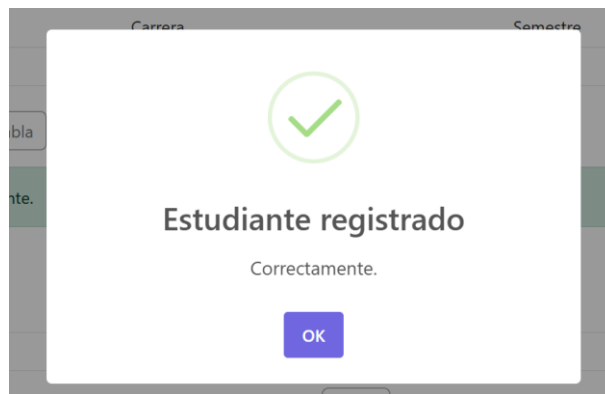
Filtro por carrera	Filtra por coincidencia exacta	Correcto	Sensible a mayúsculas y espacios, sin normalizar el texto
Limpieza general	Reinicia arreglo, tabla, contador y formulario	Correcto	No quedaron estados de edición "colgados" tras reiniciar

10. GRÁFICOS O FOTOGRAFÍAS:

7.31 Probar el sistema con al menos cinco estudiantes. El estudiante debe comprobar el registro, JSON individual, JSON general, edición, eliminación, limpieza y actualización del contador.

Ingreso de estudiantes

Figura 5 SweetAlert2 Registrado



Descripción: Se muestra la alerta de SweetAlert2 que confirma el registro exitoso de un estudiante.

Figura 6 Estudiantes registrados

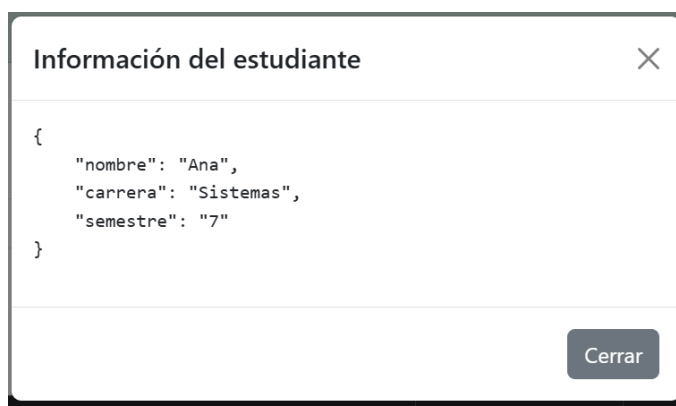
Total de estudiantes registrados: 5

#	Nombre	Carrera	Semestre	Acciones
1	ana	Tecnologías de la Información	4	ver editar eliminar
2	Ana	Sistemas	7	ver editar eliminar
3	Miguel	Sistemas	5	ver editar eliminar
4	Mishell	Software	7	ver editar eliminar
5	Pedro	Software	2	ver editar eliminar

Descripción: Se visualiza la tabla con los estudiantes registrados durante la prueba del sistema.

JSON individual

Figura 7 JSON individual



Descripción: Se muestra el modal con el JSON individual del estudiante seleccionado.

JSON general

Figura 8 JSON cursos



Descripción: Se muestra el modal con el JSON de todos los estudiantes registrados en el arreglo cursos.

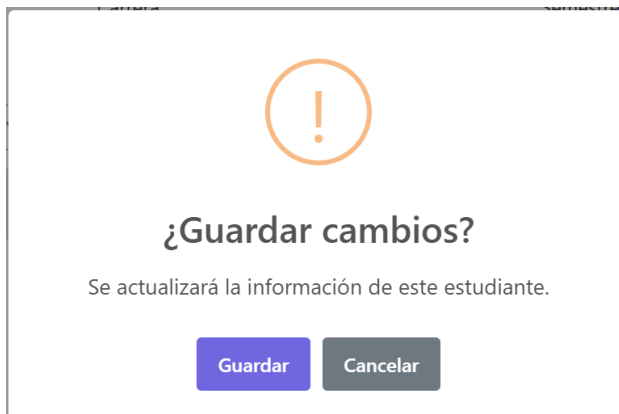
Edición

Figura 9 Editar

Nombre	Carrera	Semestre
Mishell	Software	7
Guardar cambios	Limpiar tabla	Ver JSON General
Eliminar Todo		

Descripción: Se visualiza el formulario cargado con los datos del estudiante en modo edición.

Figura 10 Sweetalert2 editar



Descripción: Se muestra la ventana de SweetAlert2 confirmando el guardado de los cambios de edición.

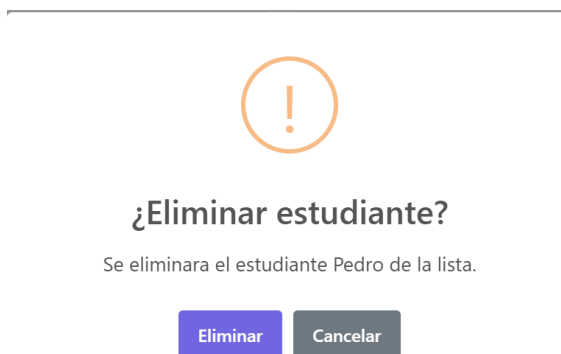
Figura 11 Tabla editada

#	Nombre	Carrera	Semestre	Acciones
1	ana	Tecnologías de la Información	4	ver editar eliminar
2	Ana	Sistemas	7	ver editar eliminar
3	Miguel	Sistemas	5	ver editar eliminar
4	Elking	Software	4	ver editar eliminar
5	Pedro	Software	2	ver editar eliminar

Descripción: Se visualiza la tabla actualizada con la información del estudiante editado.

Eliminación

Figura 12 sweetalert2 eliminar?



Descripción: Se muestra la ventana de SweetAlert2 solicitando confirmación para eliminar al estudiante.

Figura 13 Eliminado de tabla

Total de estudiantes registrados: 4

#	Nombre	Carrera	Semestre	Acciones
1	ana	Tecnologías de la Información	4	<button>ver</button> <button>editar</button> <button>eliminar</button>
2	Ana	Sistemas	7	<button>ver</button> <button>editar</button> <button>eliminar</button>
3	Miguel	Sistemas	5	<button>ver</button> <button>editar</button> <button>eliminar</button>
4	Elking	Software	4	<button>ver</button> <button>editar</button> <button>eliminar</button>

Descripción: Se visualiza la tabla sin el registro del estudiante que fue eliminado.

limpieza y actualización del contador

Figura 14 Actualiza contador

Total de estudiantes registrados: 4

Descripción: Se muestra el contador actualizado con la cantidad correcta de estudiantes registrados.

Figura 15 sweetalert reinicio



¿Eliminar todo?

Se perderan todos los datos ingresados.

Reiniciar

Cancelar

Descripción: Se visualiza la confirmación de SweetAlert2 previa al reinicio total de los datos.

Figura 16 Limpieza total

Ingrese los datos del estudiante y agréguelo dinámicamente a la tabla.

Nombre Carrera Semestre
Ejemplo: Ana Torres Seleccione...
Agregar estudiante Limpiar tabla Ver JSON General Eliminar Todo

Estudiante eliminado correctamente.

Buscar

Nombre,carrera o semestre Carrera
Ejemplo: Ana Torres Todas
Buscar Limpiar búsqueda Filtrar

Total de estudiantes registrados: 0

#	Nombre	Carrera	Semestre	Acciones
---	--------	---------	----------	----------

Descripción: Se muestra la tabla, el arreglo y el formulario completamente vacíos tras la limpieza total.

11. DISCUSIÓN:

Como usuario común, normalmente no se está acostumbrado a ver la información en JSON, ya que las aplicaciones suelen mostrar los datos ya procesados y presentados de forma amigable. Sin embargo, es importante trabajar con ellos ya que detrás de cualquier sistema, especialmente al consumir APIs, se trabajó con JSON inmensos

Al asignar el evento **click** a los botones de eliminar de cada fila. Inicialmente se utilizó `eliminar.addEventListener("click", eliminarEstudiante(indice));`, pero esta instrucción ejecutaba la función al crear el botón en lugar de hacerlo cuando el usuario hacía clic, ya que los paréntesis provocan su ejecución inmediata y `addEventListener` recibe el valor de retorno. La solución consistió en envolver la llamada en una función flecha: `eliminar.addEventListener("click", () => eliminarEstudiante(indice));`, logrando que `eliminarEstudiante(indice)` se ejecute únicamente al producirse el evento y permitiendo enviar el índice del estudiante correspondiente

La función `agregarFilaTabla` borra por completo el contenido del cuerpo de la tabla con `cuerpoTabla.innerHTML = ""` y luego vuelve a crear todas las filas desde cero recorriendo el arreglo completo, incluso cuando solo se agregó, editó o eliminó un único estudiante. En un proyecto pequeño como este, con pocos registros, el impacto no se nota, pero en un sistema con cientos o miles de datos, reconstruir toda la tabla en cada acción representaría un costo de rendimiento innecesario, ya que se estarían eliminando y creando nodos del DOM que ni siquiera cambiaron

12. CONCLUSIONES:

- Registrar a los estudiantes en un arreglo de objetos y actualizar la tabla dinámicamente cada vez que se agregaba un nuevo registro, usando el arreglo con `forEach` y creando los elementos de la tabla mediante `createElement` y `appendChild` permitió comprender que el DOM no solo sirve para mostrar información fija, sino que puede reconstruirse por completo a partir de los datos almacenados en JavaScript
- Al construir un modal de Bootstrap que muestra tanto el JSON individual de un estudiante como el JSON general de todos los registros, utilizando `JSON.stringify` para convertir los objetos en texto legible entender cómo transformar datos internos de JavaScript en una representación clara para el usuario, se logra apreciar la importancia de modal como pantallas emergentes

- Las funciones de edición y eliminación de estudiantes, incluyendo el modo de edición para diferenciar entre agregar y modificar un registro, así como la actualización del contador y del arreglo tras cada acción. Con esto se reforzó la importancia de mantener sincronizados el arreglo de datos y la interfaz visual, ya que cualquier cambio en uno debe reflejarse correctamente en el otro para evitar inconsistencias en la información mostrada.
- Se implementaron el buscador por texto y el filtro por carrera, realizando las comparaciones directamente sobre el arreglo de estudiantes y no sobre las filas ya dibujadas, y se integró SweetAlert2 para mejorar los mensajes de confirmación y notificación. Esta parte permitió comprender la diferencia entre filtrar datos y filtrar elementos visuales, además de valorar cómo una librería externa puede mejorar significativamente la experiencia del usuario frente a los mensajes básicos del navegador.

13. BIBLIOGRAFÍA:

Mozilla Contributors. (2024). MDN Web Docs: Glosario de términos relacionados con la Web y JavaScript. Mozilla Foundation. Recuperado de <https://developer.mozilla.org/es/docs/Glossary>

Haverbeke, M. (2018). Eloquent JavaScript: A Modern Introduction to Programming (3rd ed.). No Starch Press.

W3Schools. (s.f.). w3schools. Obtenido de 3schools.com: <https://www.w3schools.com/js/>

Alessandra Salvaggio, G. T. (2019). JavaScript: Guía completa. España: Marcombo.

14. Criterio personal

Gracias a este laboratorio comprendo que el DOM es un árbol para que el motor del navegador pueda construir nuestra página web y la manipulación es muy importante ya que gracias a ella podemos “editar” el archivo .html en tiempo de ejecución, esto con la cantidad de eventos que hay para trabajar con variables que son elementos del .html creadas hacen que las funcionalidades que podemos crear sean infinitas.

Los modales y las librerías externas utilizados fueron de gran utilidad al momento de mostrar mensajes para garantizar la experiencia inmersiva, además de darle una capa de confirmación al usuario para evitar errores.

Este fue mi primer CRUD en el lenguaje JavaScript fue algo confuso al inicio, pero poco a poco se va comprendiendo